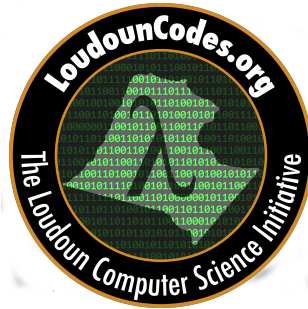




Karel

Creative Project Labs

Colors, walls, and events
with LoudounCodes JKarel

LAB 01 · TEAM TRAILS
LAB 02 · EVENT SCOREBOARD

For students ready to make their own maze,
game, simulation, or artwork.

Java 18 · jGRASP · JKarel 1.0.0
Classroom review edition · October 2026



Start here: jGRASP setup

These two original labs are for students who have finished their Karel sequence and are building a game, maze, simulation, or artwork. Start with a working small program, predict its behavior, then choose a change that fits your own project. Allow one class period per lab, with extra time for the project extensions.

1. Team trails: colors and walls
2. Event scoreboard: interfaces and observers

Set up in jGRASP

Use JDK 18 or later. The library targets Java 18; jGRASP's own minimum Java version is a separate requirement. The teacher builds once with ant and gives students `out/jkarel-1.0.0.jar`. Students do not need Ant or JUnit to run these labs.

In jGRASP, open **Settings** → **PATH/CLASSPATH** → **Workspace**, select **CLASSPATH**, then **New** and browse to that JAR. Apply the setting. If your classroom uses jGRASP projects, choose the project scope instead of Workspace. Remove an older JKarel JAR entry so the compiler and runner see the same library. Keep the original FCPS library available for assignments that still import `edu.fcps.karel2`; these labs import `org.loudouncodes.jkarel`.

Copy the example `.java` file into your own working folder, open it in jGRASP, compile, and run. The file name must match its public class name. No map files or extra dependency JARs are needed for these two examples.

- package `org.loudouncodes.jkarel` does not exist: check the CLASSPATH entry.
- `UnsupportedClassVersionError`: check the Java used to run in jGRASP; it needs 18+.
- A wall exception: read the failing line and use the appropriate `...IsClear()` sensor before that move. The robot remains where it was.
- STEP pacing waits for Enter in the run console. FAST is used in the examples.

Teacher command-line equivalent, from the repository root:

```
ant
javac --release 18 -cp out/jkarel-1.0.0.jar -d build/labs examples/java/TeamTrails.java \
    examples/java/EventScoreboard.java
java -cp out/jkarel-1.0.0.jar:build/labs TeamTrails
java -cp out/jkarel-1.0.0.jar:build/labs EventScoreboard
```

On Windows, replace the runtime classpath's `:` separator with `;`. Close each arena window before running the next example.



Extension 1: Team trails

How could a player tell which team owns a trail? Build a small arena with two colored robots, then use the colors and walls in a project of your choice.

You should already be comfortable constructing a Robot, calling its methods, using loops, and checking a sensor. After this lab you should be able to create walls in code and explain how a robot's color affects its dropped beepers.

Predict, run, inspect

Open `TeamTrails.java` using the [jGRASP setup](#). Before running, sketch the grid. Predict the robots' final locations, their remaining inventories, and the colors of the beepers. Run it and compare with your sketch.

Expected result: a red trail on row 2 and a blue trail on row 4, each covering columns 2 through 6. Both robots end in column 6, with no beepers left. A wall separates the lanes. The program builds its own outer boundary.

Color comes from `java.awt`. Use `Color.RED`, `Color.BLUE`, or `new Color(80, 160, 220)`; each red/green/blue component is an integer from 0 to 255.

```
robot.setColor(Color.BLUE);  
robot.putBeeper();
```

The drop colors the **whole stack** at that location. If a red robot later drops onto a blue stack, the stack becomes red. Picking up a beeper preserves the color of the remaining stack. Picking it up does not change the robot's color.

`Arena.addNorthWall(x, y)` places a unit wall immediately north of that cell. The `East`, `South`, and `West` methods work the same way. Adjacent calls can build a longer wall. `Arena.setSize` changes the displayed grid; it does not enclose it with a full boundary. Build boundaries explicitly, as this example does.

Make it yours

1. Change both team colors. Explain why you need an import for `Color`.
2. Change a robot's color halfway through its trail. Predict which beepers change.
3. Add a second divider with a gap. Write a robot route through the gap using turns and sensors; keep the route inside the boundary.
4. Have the robots meet and drop on the same cell. Test the whole-stack color rule.

Choose one extension for your project: colored territory, breadcrumbs through a maze, team bases, or a drawing made by several robots. Color is appearance, not a team identifier: keep a separate variable or robot reference for game rules.

Show what you learned

Show your arena and the code that builds it. Explain a north-wall coordinate, why a move is safe, and the result of two different colored drops on one cell. Give a prediction and observation for one change you made. Your program should finish its planned route without a wall exception or an empty-inventory drop.

Teacher prompts: ask why the loop avoids moving after the final drop; have a student point to the wall that a left-facing sensor would detect. Extra challenge: extract a reusable `buildRoom` method that takes a location and dimensions.

Extension 2: Let the arena tell the scoreboard

A game needs to count pickups and moves. Should every robot have to know about the scoreboard? Use an interface to let a separate observer react to arena events.

Complete Team trails first. You should know fields, methods, conditionals, and object references. This lab introduces an interface with default methods and the observer pattern: one object performs an action; another receives a notification.

Predict and run

Open `EventScoreboard.java` using the [jGRASP setup](#). Read `takeTurn` and `Scoreboard`. Predict the number of pickup callbacks and move callbacks before running.

Expected console results progress through red/blue scores of 1/0, 1/1, 2/1, and 2/2. The final line is `Final: Red 2, Blue 2, moves 8`. Both robots finish at column 6; each holds two beepers. The main loop drives the turns. The scoreboard never moves a robot.

```
Arena.addListener(scores);
```

This registers an object implementing `ArenaListener`. Its default methods do nothing, so the class overrides only the events it wants. `@Override` lets the compiler check the method name and parameter type. For a pickup the library calls `beeperPickedUp(Robot robot)` after updating the stack and inventory. Comparing `robot == red` tests whether this is the exact red robot object; its appearance could change while its identity stays the same.

Experiments

1. Move one beeper farther down its lane. Predict which turn produces its pickup.
2. Count drops separately by overriding `beeperDropped`. Add a drop in `takeTurn` after a pickup. Watch the scores: decide whether a pickup of a previously dropped beeper should award another point in your game.
3. Add a third robot and track it explicitly. Color alone must not determine score.
4. Remove the listener before the final turn. Explain why the robot still moves but that action no longer changes the observed counters.

Choose a project use: count maze steps, track food pickups, record a move log, or announce when a player has collected enough items. Write the win check in the main game loop after taking a turn; the callback can update a flag or counter.



Event scoreboard: callback rules

- Callbacks run immediately on the thread that performs the action. Keep them short. Do not start another movement or a game loop from a callback.
- `robotMoved` reports completed forward/backward movement; turns have no event.
- A robot drop produces `beeperAdded` followed by `beeperDropped`. Use the latter to count player drops; counting both would count one action twice.
- `robotAdded` runs before the constructor returns. Register scoring after player creation, as this example does; do not depend on subclass fields in that event.
- `wallCollision` observes an attempted blocked move before an exception is thrown. Sensors are still the right way to plan safe movement.
- `mapLoaded` fires after loading. Map loading preserves existing robots and listeners while replacing walls, beepers, and custom items.
- There are no keyboard-input, timer, turn, or color-change callbacks yet.

The `finally` block removes this listener even if a robot action fails. It does not reset the scoreboard, arena, or robots; those are separate responsibilities.

Show what you learned

Demonstrate one new counter or announcement. Explain the difference between calling a robot method and receiving a callback, why `pickup` sees the new inventory, and why the game loop belongs outside the listener. Include the predicted and observed callback counts for your change.

Teacher prompts: ask the student to trace one `move()` and one `pickBeeper()` into their callbacks. Extra challenge: add a second listener that records a text log; explain how it can coexist with scoring without modifying `Robot`.

Complete example programs

Copy these programs into files with the names shown. Both examples are also in the repository's examples/java folder. Compile and run each separately in jGRASP.

TeamTrails.java

```
import java.awt.Color;
import org.loudouncodes.jkarel.*;

/** Extension lab 1: two teams leave colored trails inside a student-built arena. */
public class TeamTrails {
    public static void main(String[] args) {
        Arena.openDefaultMap();
        Arena.setSize(8, 6);
        Arena.setPace(Pacing.FAST);
        buildWalls();

        Robot red = new Robot(2, 2, Direction.EAST, 5);
        Robot blue = new Robot(2, 4, Direction.EAST, 5);
        red.setColor(Color.RED);
        blue.setColor(Color.BLUE);

        for (int step = 0; step < 5; step++) {
            red.putBeeper();
            blue.putBeeper();
            if (step < 4) {
                if (red.frontIsClear()) red.move();
                if (blue.frontIsClear()) blue.move();
            }
        }
        System.out.println("Both teams left five beepers, from column 2 through column 6.");
    }

    private static void buildWalls() {
        // setSize changes the visible grid. These loops create the boundaries.
        for (int x = 1; x <= 8; x++) {
            Arena.addSouthWall(x, 1);
            Arena.addNorthWall(x, 6);
        }
        for (int y = 1; y <= 6; y++) {
            Arena.addWestWall(1, y);
            Arena.addEastWall(8, y);
        }
        // A divider between the two teams' lanes.
        for (int x = 2; x <= 6; x++) Arena.addNorthWall(x, 2);
    }
}
```



EventScoreboard.java

```
import java.awt.Color;
import org.loudouncodes.jkarel.*;

/** Extension lab 2: observe moves and pickups without putting scoring in Robot. */
public class EventScoreboard {
    public static void main(String[] args) {
        Arena.openDefaultMap();
        Arena.setSize(8, 6);
        Arena.setPace(Pacing.FAST);
        for (int x = 2; x <= 6; x++) Arena.addNorthWall(x, 2);
        Arena.addEastWall(6, 2);
        Arena.addEastWall(6, 4);
        Arena.addBeeper(3, 2); Arena.addBeeper(4, 2);
        Arena.addBeeper(3, 4); Arena.addBeeper(4, 4);

        Robot red = new Robot(2, 2, Direction.EAST, 0);
        Robot blue = new Robot(2, 4, Direction.EAST, 0);
        red.setColor(Color.RED); blue.setColor(Color.BLUE);
        Scoreboard scores = new Scoreboard(red, blue);
        Arena.addListener(scores);
        try {
            // The main program controls movement; the listener only observes it.
            for (int turn = 0; turn < 4; turn++) {
                takeTurn(red);
                takeTurn(blue);
            }
            scores.printResult();
        } finally {
            Arena.removeListener(scores);
        }
    }

    private static void takeTurn(Robot robot) {
        if (robot.frontIsClear()) robot.move();
        if (robot.nextToABeeper()) robot.pickBeeper();
    }

    private static class Scoreboard implements ArenaListener {
        private final Robot red;
        private final Robot blue;
        private int redScore;
        private int blueScore;
        private int moves;

        Scoreboard(Robot red, Robot blue) {
            this.red = red;
            this.blue = blue;
        }

        @Override
        public void robotMoved(Robot robot) {
            if (robot == red || robot == blue) moves++;
        }

        @Override
        public void beeperPickedUp(Robot robot) {
            if (robot == red) redScore++;
            if (robot == blue) blueScore++;
            System.out.println("Red: " + redScore + " Blue: " + blueScore);
        }

        void printResult() {
            System.out.println("Final: Red " + redScore + ", Blue " + blueScore
                + ", moves " + moves);
        }
    }
}
```



Teacher notes and sources

Teacher acceptance

Before giving the JAR to students, run both examples in the actual classroom jGRASP installation. Check red and blue robots/trails in the first lab; check pickup scores and the final eight-move count in the second. Java installation and visual display are classroom checks, separate from the automated Ant tests.

Sources and reuse

The exercises and example programs were written for this repository; they do not adapt the restricted Fairfax worksheets. The library's Fairfax/Loudoun ancestry remains documented in the root README. These files use the repository's existing [GPL version 3 license](#); no separate curriculum license is introduced by these labs. A broader curriculum publication can make that choice explicitly later.

The jGRASP setup instructions are independently written from the official [jGRASP installation tutorial, chapter 1, PATH/CLASSPATH section](#) and [project tutorial, chapter 7](#), version 1.8.7 (2009), accessed 2026-10-08. These are citations for setup facts; no tutorial text or figures are reproduced. Menus may vary with classroom version.

Brand asset: the existing LoudounCodes logo supplied from the personal Dropbox Marketing/logos collection. PDF layout and examples are created for this handout.